

컴퓨터 비전

Computer Vision

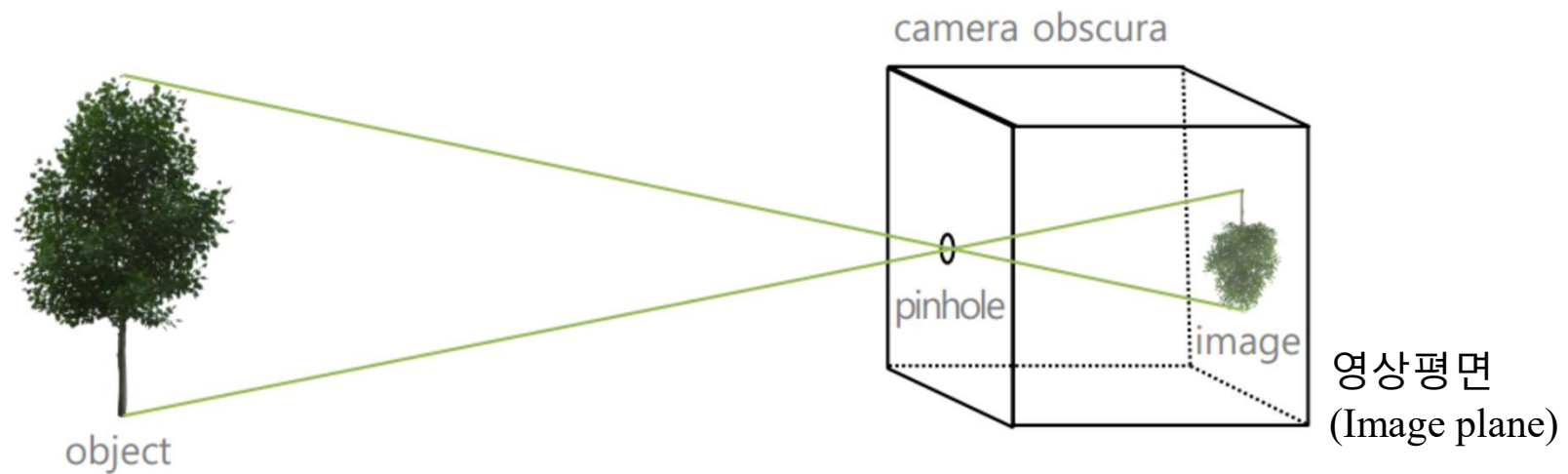
- Stereo Vision -

동아대학교 소프트웨어대학 AI학과
2026년 1학기

임 한 신

카메라 모델

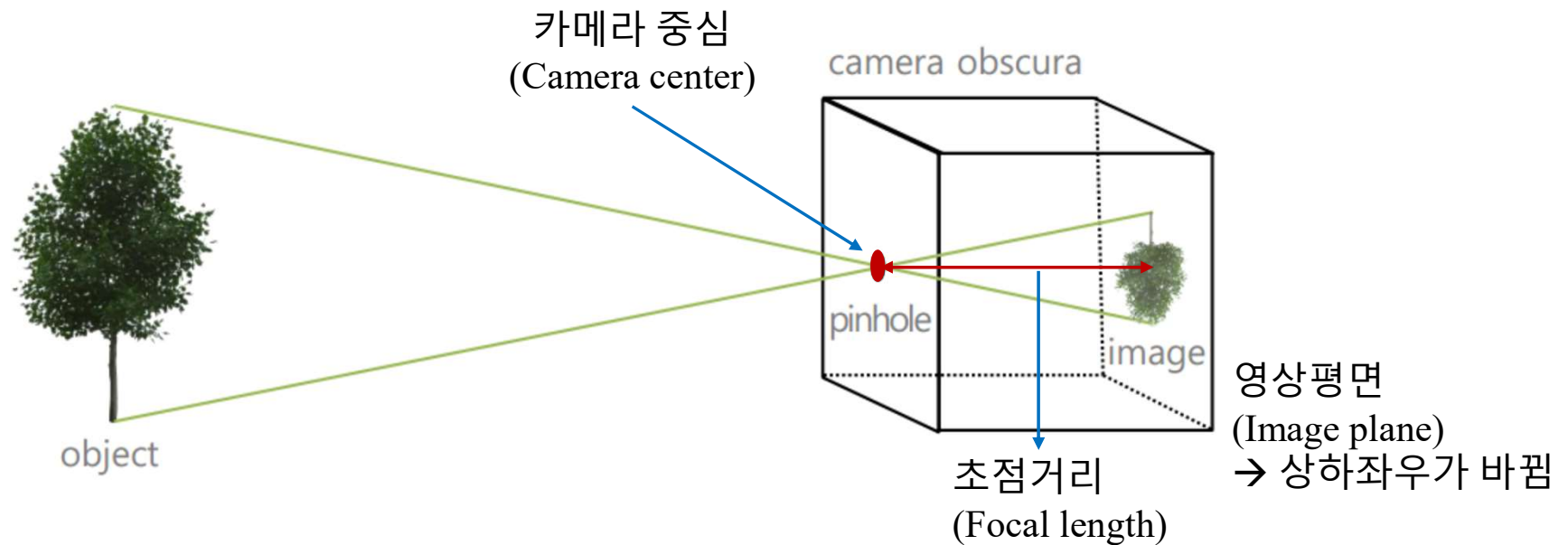
- 핀홀 카메라 모델(Pinhole camera model)



물체에서 나온 빛은 pinhole을 일직선으로 통과하여 반대편 벽(영상평면)에 상(image)을 맺힘

카메라 모델

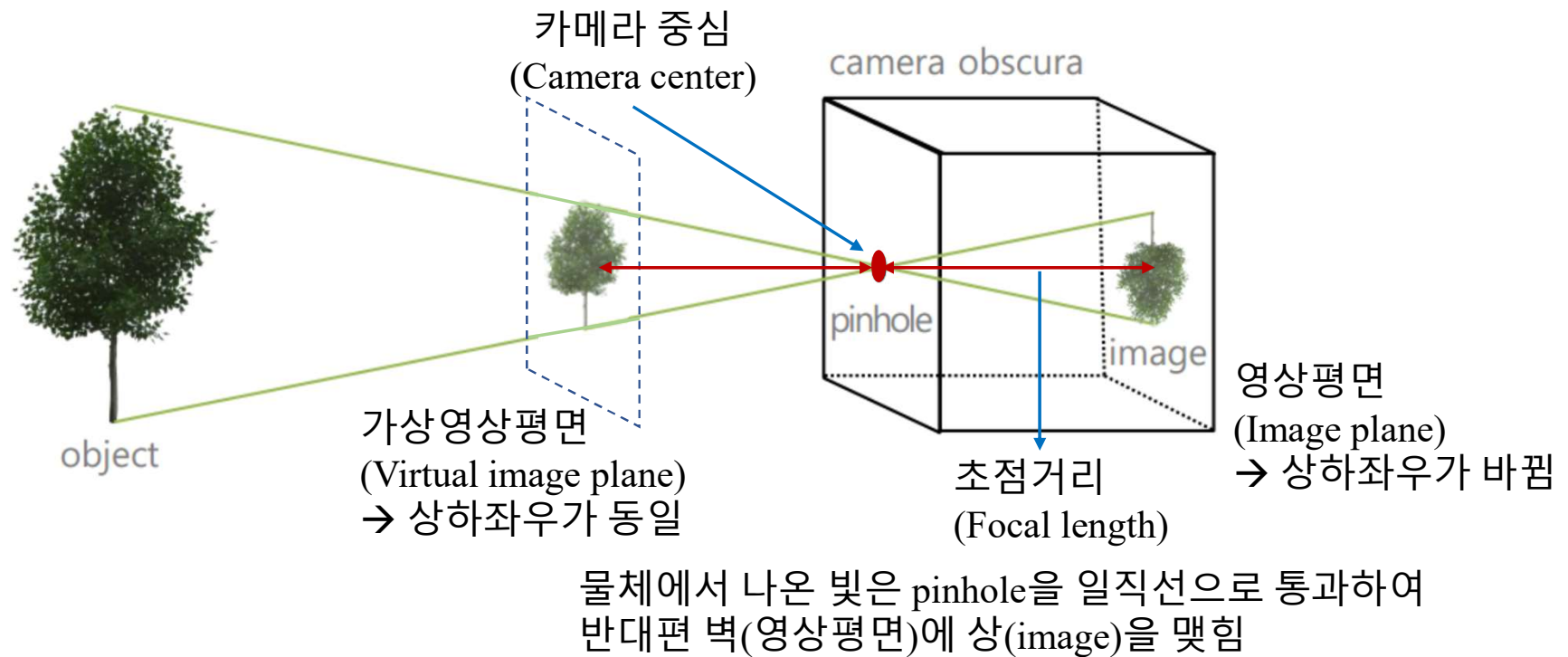
- 핀홀 카메라 모델(Pinhole camera model)



물체에서 나온 빛은 pinhole을 일직선으로 통과하여 반대편 벽(영상평면)에 상(image)을 맺힘

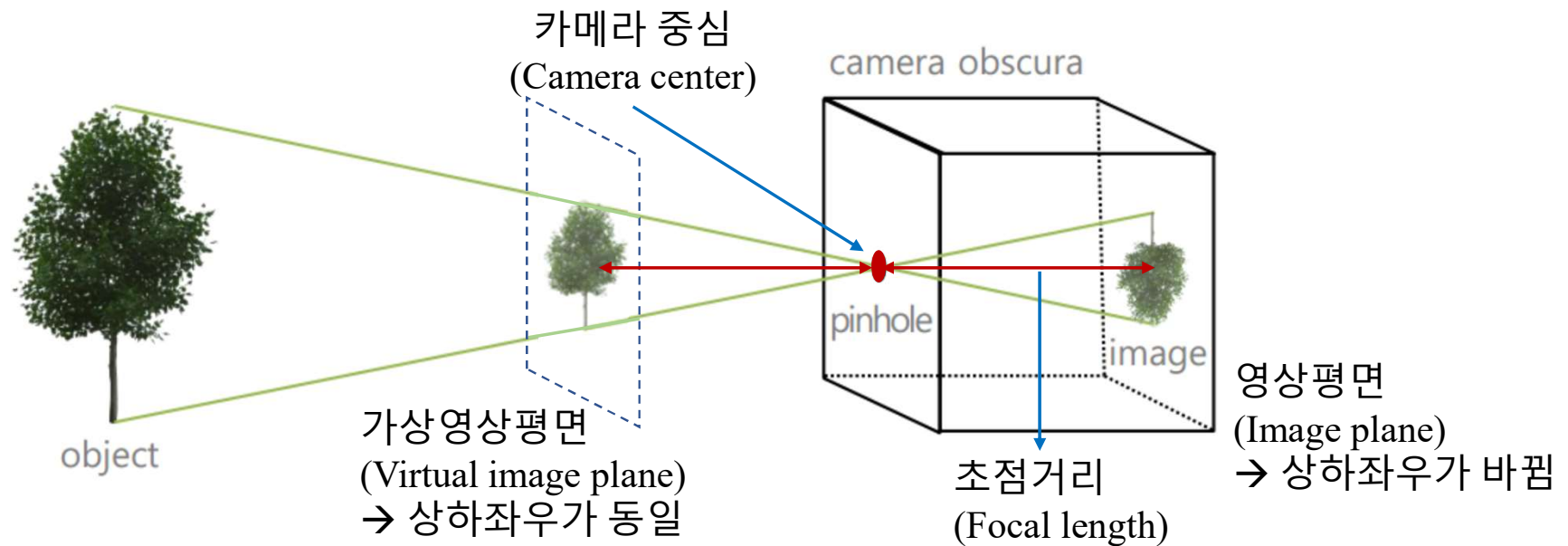
카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



카메라 모델

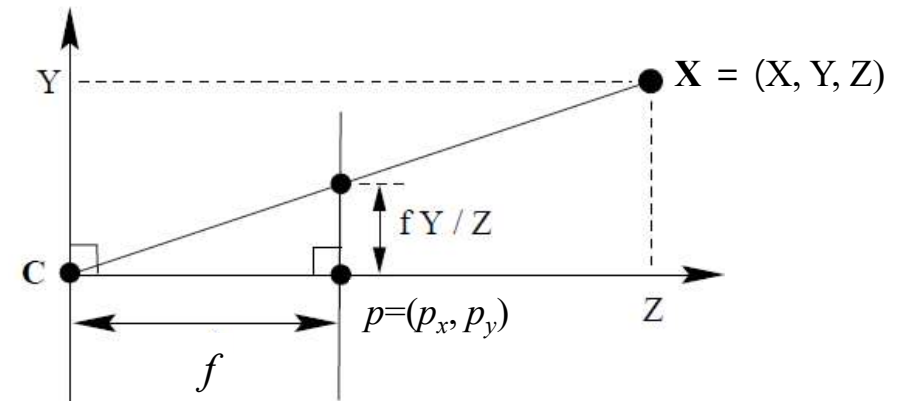
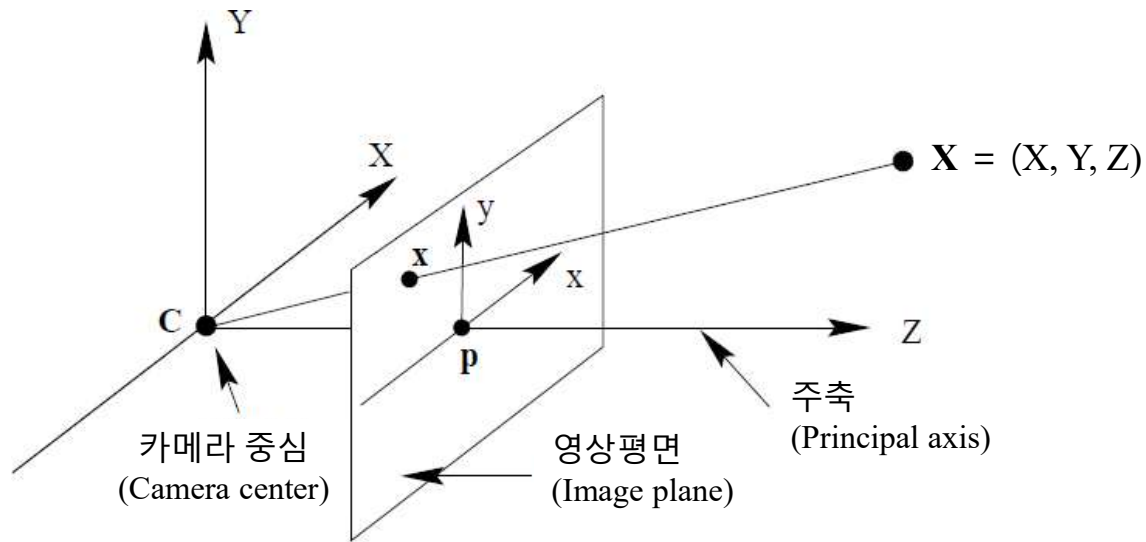
- 핀홀 카메라 모델(Pinhole camera model)



물체에서 나온 빛은 pinhole을 일직선으로 통과하여 반대편 벽(영상평면)에 상(image)을 맺힘
가상영상평면을 두면 상하좌우가 바뀌지 않으면서 영상평면과 동일한 기하학적 모델이 가능

카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(z -축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

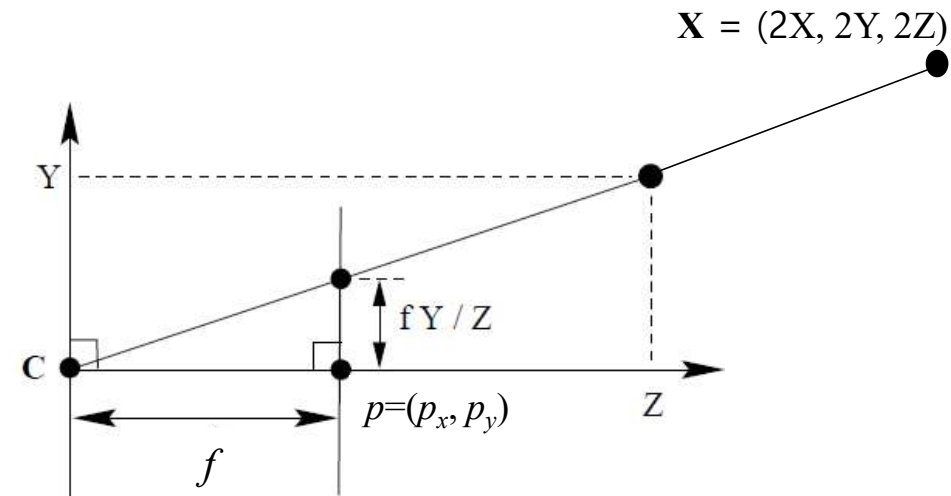
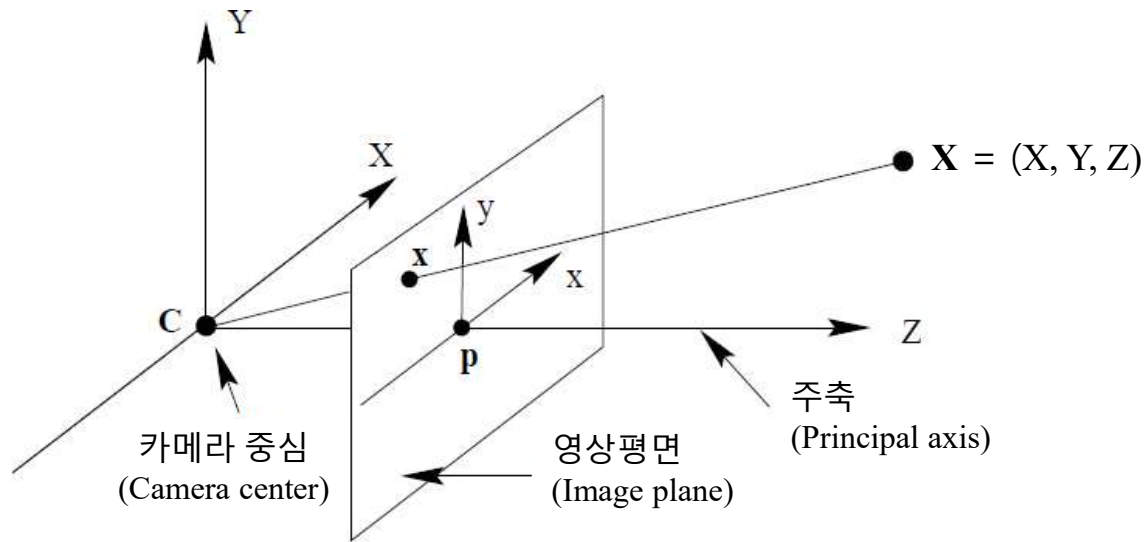
$\mathbf{x} = (x, y)$ 는 아래와 같이 계산됨

$$x = \frac{fX}{Z} + p_x$$

$$y = \frac{fY}{Z} + p_y$$

카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(z -축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

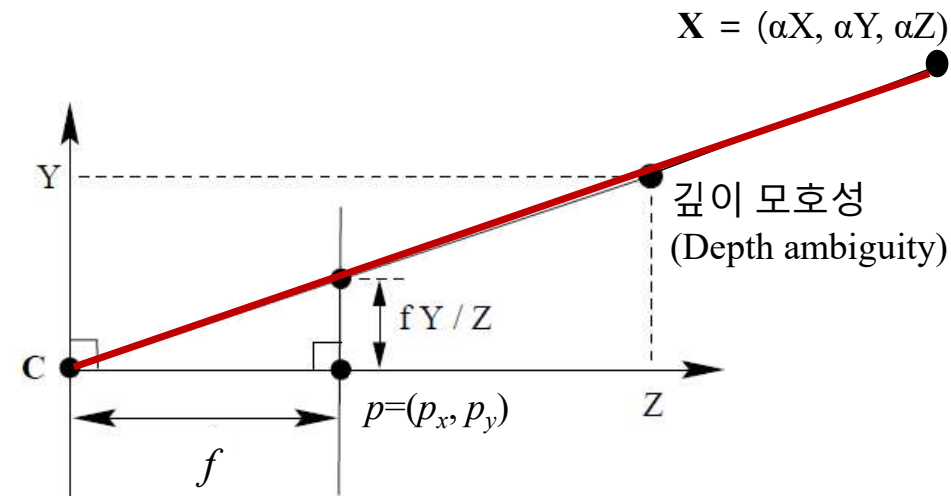
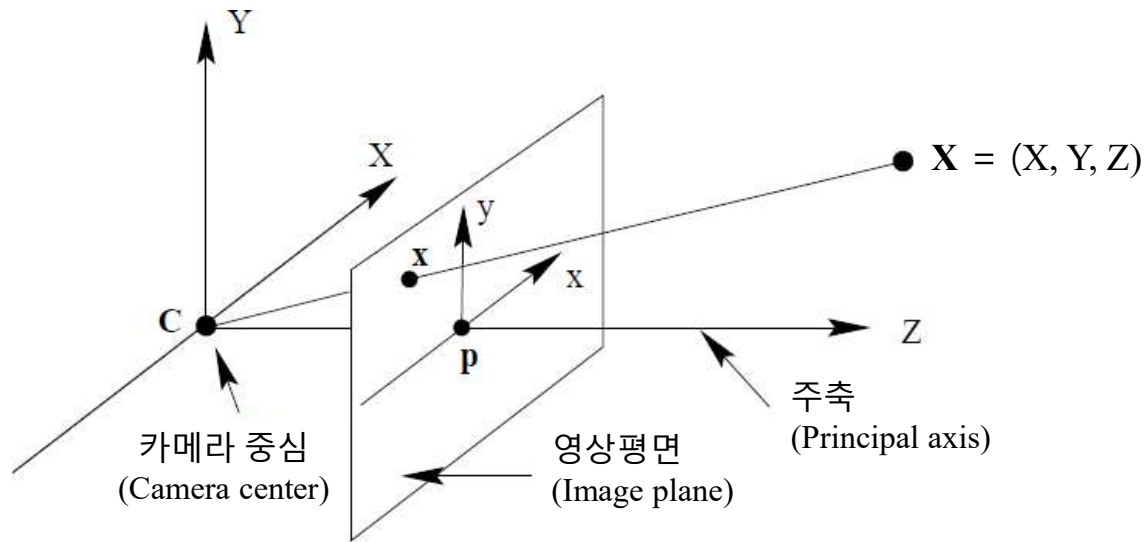
$\mathbf{x} = (x, y)$ 는 아래와 같이 계산됨

$$x = \frac{f2X}{2Z} + p_x = \frac{fX}{Z} + p_x$$

$$y = \frac{f2Y}{2Z} + p_y = \frac{fY}{Z} + p_y$$

카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(z-축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

$\mathbf{x} = (x, y)$ 는 아래와 같이 계산됨

$$x = \frac{f\alpha X}{\alpha Z} + p_x = \frac{fX}{Z} + p_x$$

$$y = \frac{f\alpha Y}{\alpha Z} + p_y = \frac{fY}{Z} + p_y$$

카메라 모델

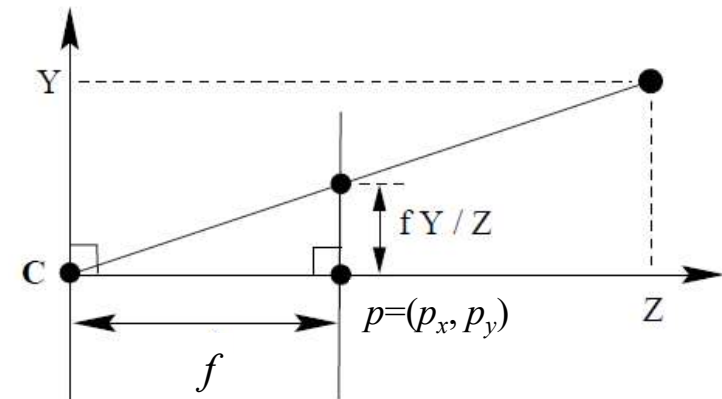
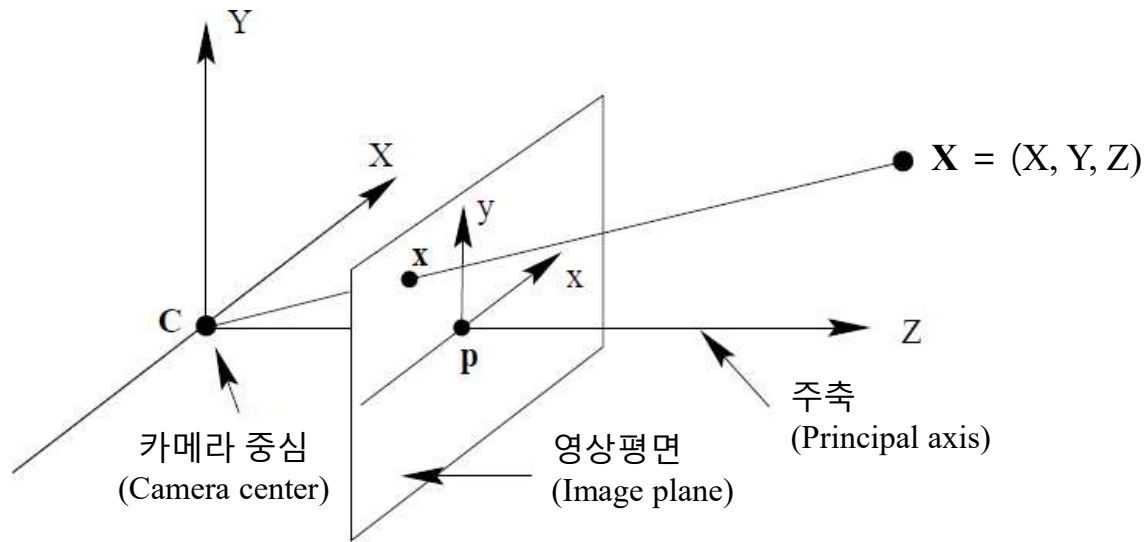
- 핀홀 카메라 모델(Pinhole camera model)



깊이 모호성(Depth ambiguity)

카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



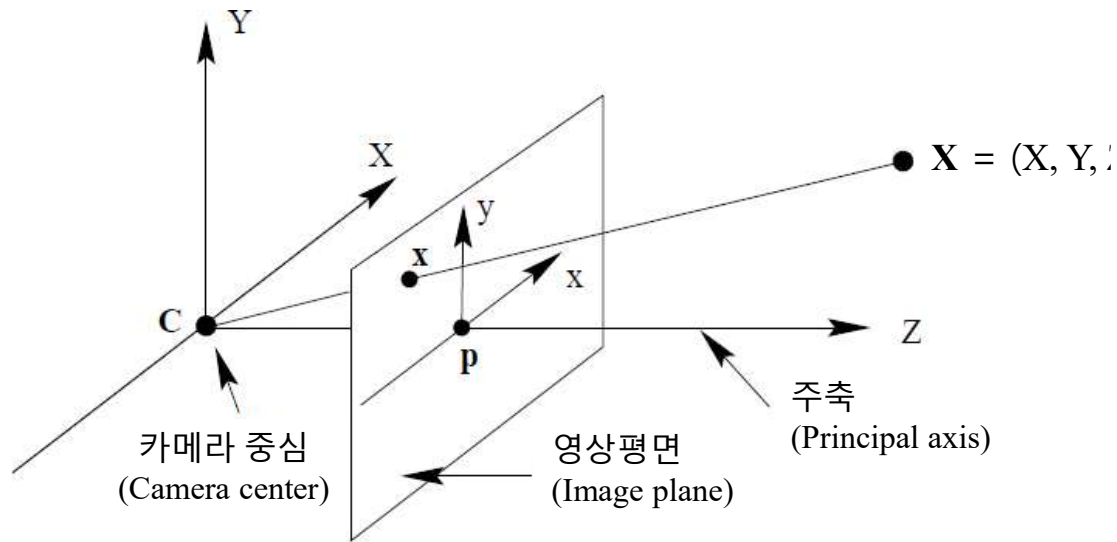
- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(z-축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

카메라 내부파라미터

- 초점거리(Focal length(단위 : pixels))
- 주점(Principal point(단위 : pixels))

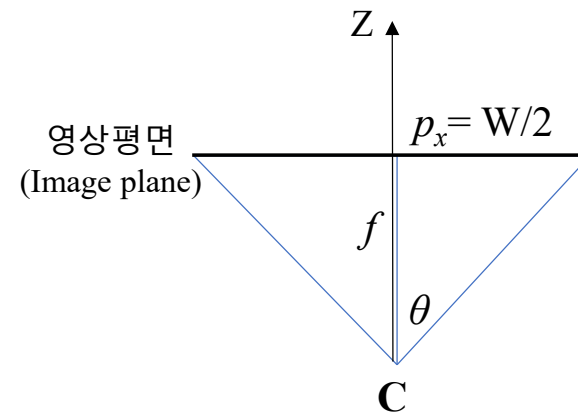
카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(z-축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

카메라 화각 계산



$$\text{수평 화각} = 2\theta = 2 \times \arctan(W / 2f)$$

예) $W : 1920, f : \text{약 } 1330$ 라고 하면

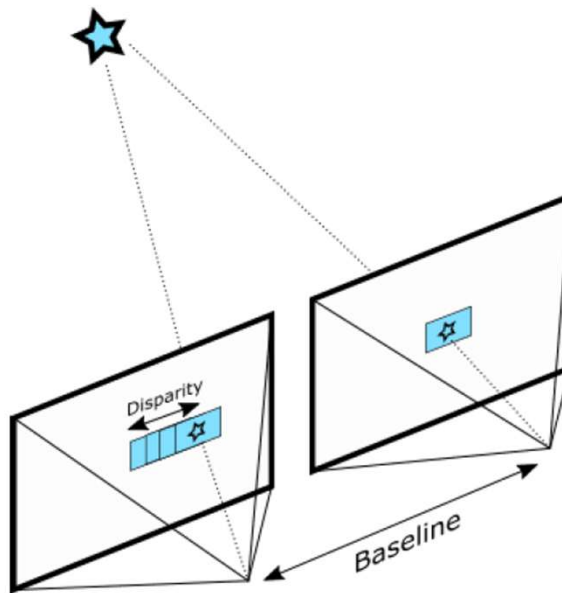
수평 화각은 약 71.6도 정도 나옴

스테레오 비전(Stereo Vision)

- 두 개의 카메라(스테레오 카메라)를 이용해 마치 사람의 두 눈처럼 사물의 입체감과 거리(깊이, Depth)를 추정하는 기술
- Disparity(양안 시차) : 두 눈(또는 두 카메라) 사이의 시점 차이로 인해 발생하는 영상 간의 위치 차이
- 참고 : 인간의 눈은 약 6.5cm 떨어져 있음



스테레오 카메라



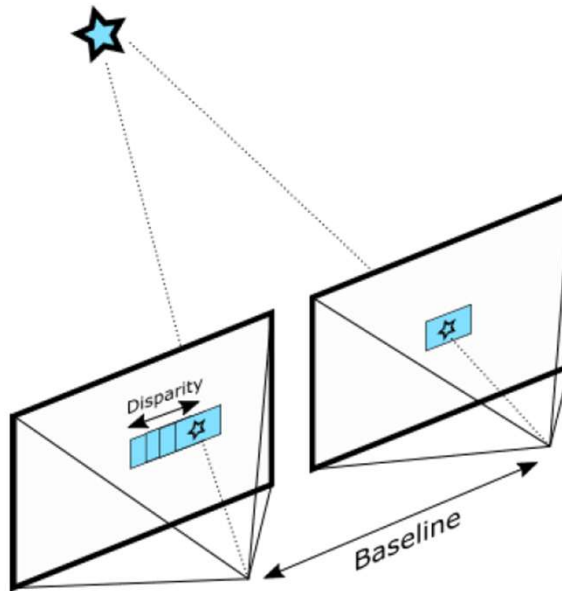
Left image

스테레오 비전(Stereo Vision)

- 두 개의 카메라(스테레오 카메라)를 이용해 마치 사람의 두 눈처럼 사물의 입체감과 거리(깊이, Depth)를 추정하는 기술
- Disparity(양안 시차) : 두 눈(또는 두 카메라) 사이의 시점 차이로 인해 발생하는 영상 간의 위치 차이
- 참고 : 인간의 눈은 약 6.5cm 떨어져 있음



스테레오 카메라



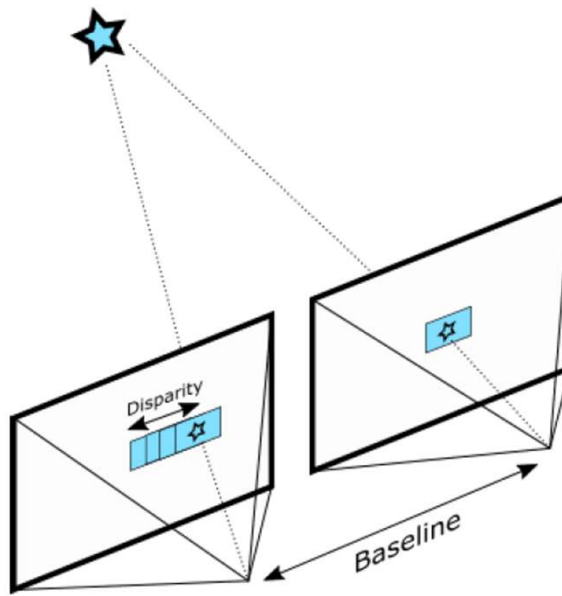
Right image

스테레오 비전(Stereo Vision)

- 두 개의 카메라(스테레오 카메라)를 이용해 마치 사람의 두 눈처럼 사물의 입체감과 거리(깊이, Depth)를 추정하는 기술
- Disparity(양안 시차) : 두 눈(또는 두 카메라) 사이의 시점 차이로 인해 발생하는 영상 간의 위치 차이
- 참고 : 인간의 눈은 약 6.5cm 떨어져 있음



스테레오 카메라



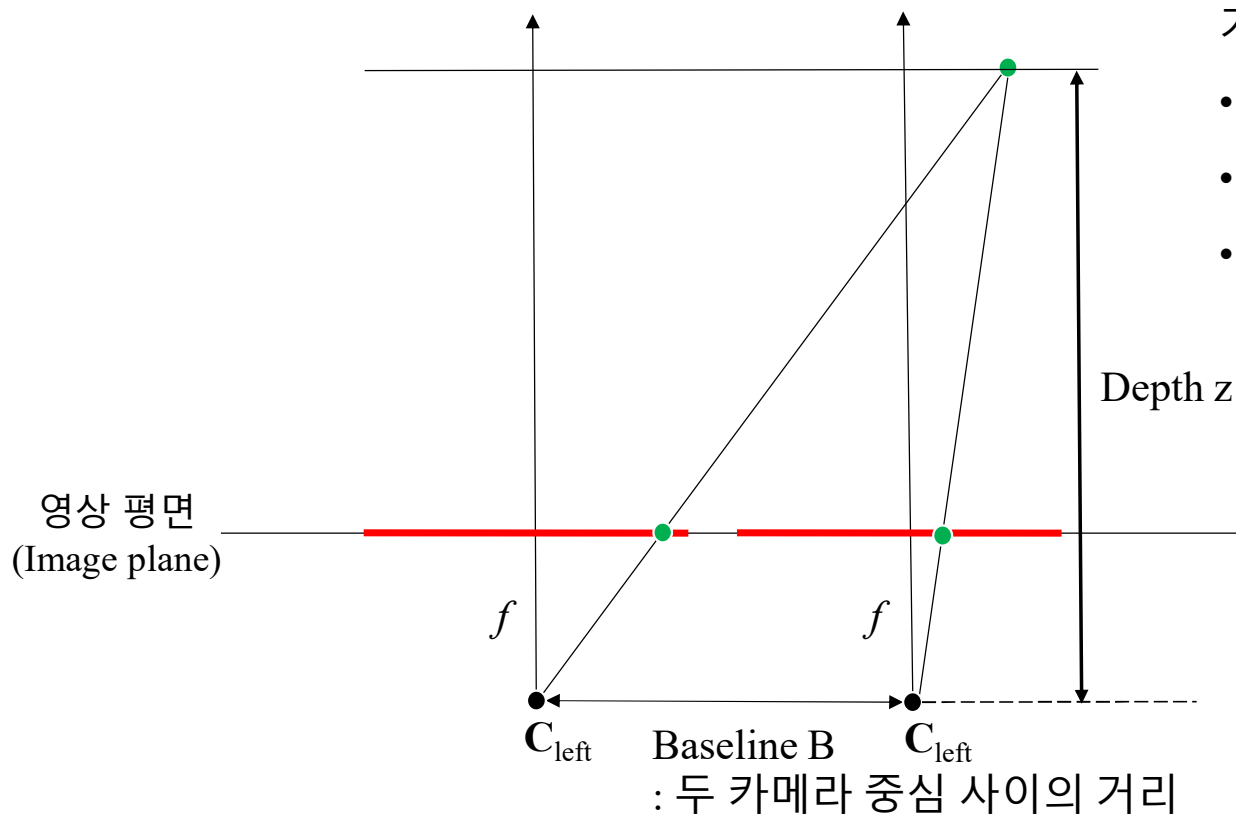
Left image



Right image

카메라 모델

- Disparity(양안시차)와 depth

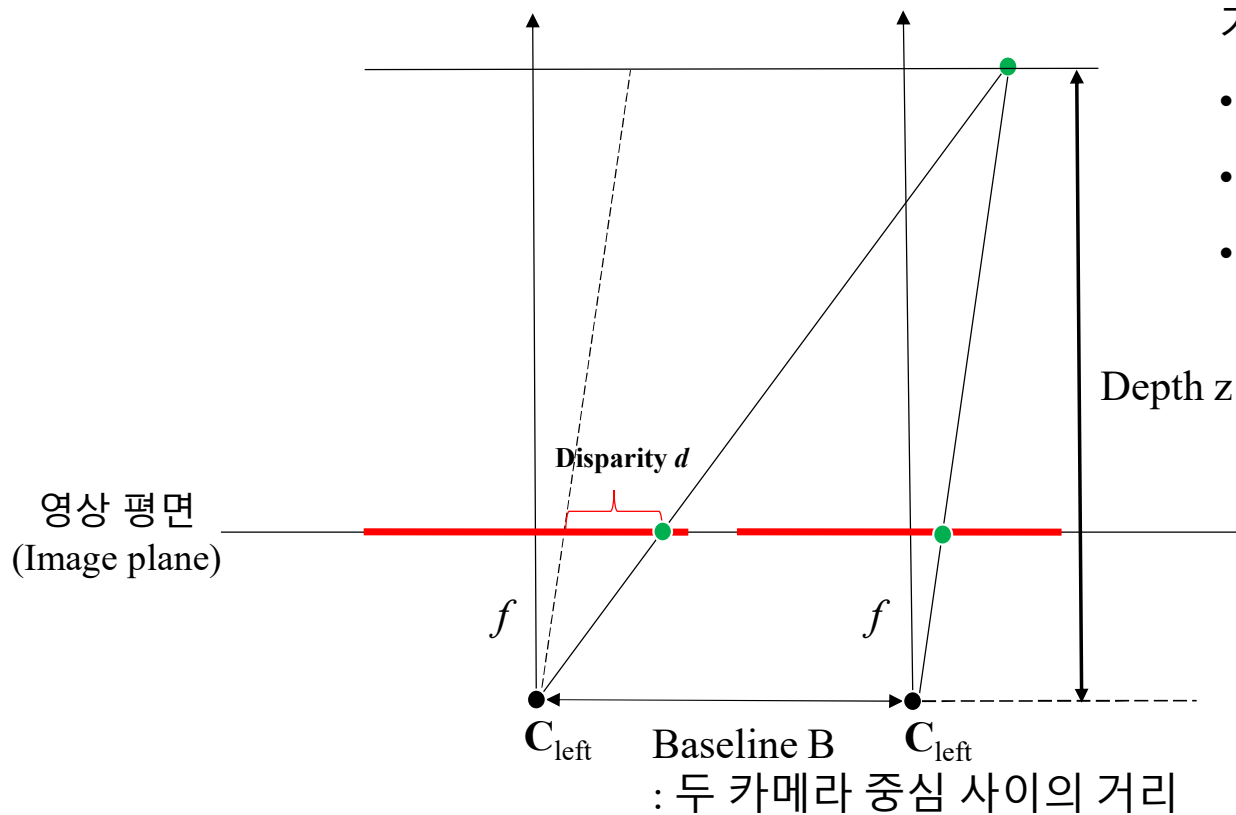


가정

- Baseline $B \neq 0$
- $f_{\text{left}} = f_{\text{right}} = f$
- 두 영상의 영상 평면은 동일한 평면 상에 있음

카메라 모델

- Disparity(양안시차)와 depth

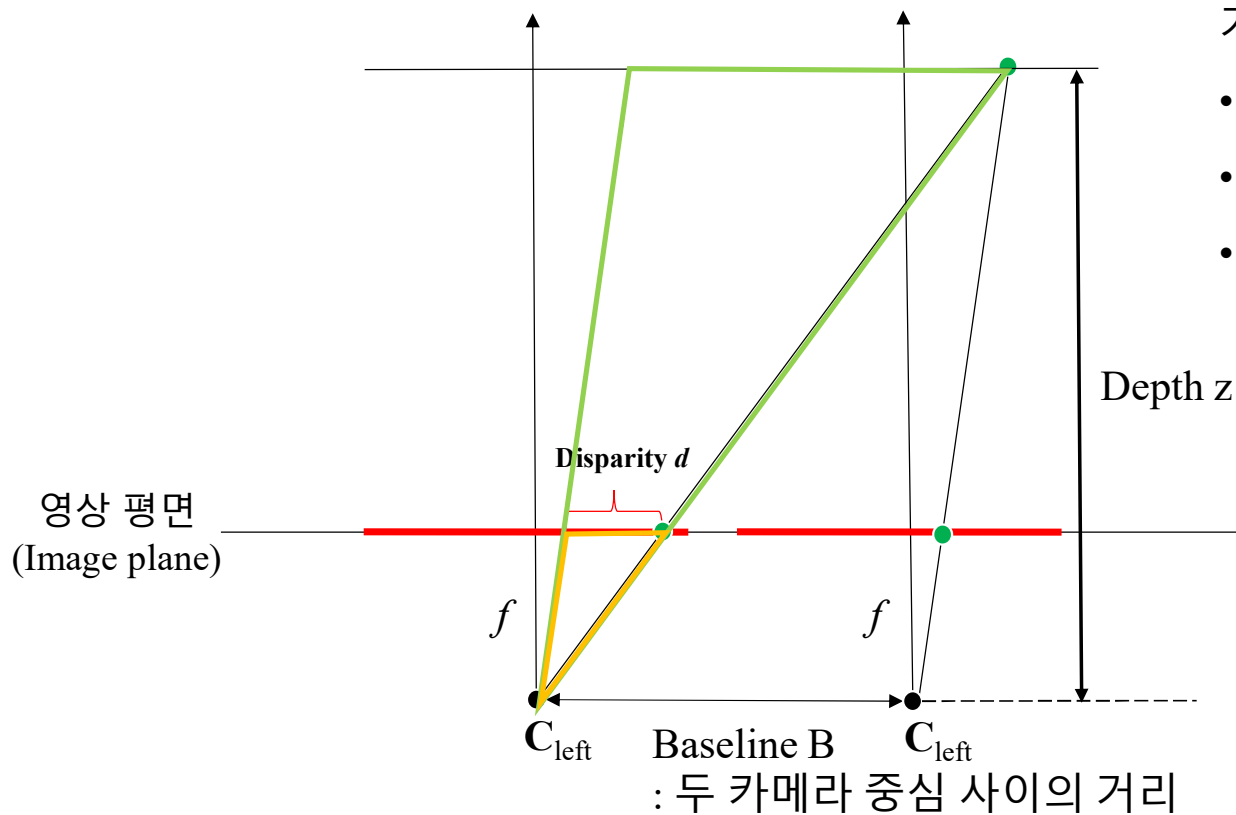


가정

- Baseline $B \neq 0$
- $f_{left} = f_{right} = f$
- 두 영상의 영상 평면은 동일한 평면 상에 있음

카메라 모델

- Disparity(양안시차)와 depth



가정

- Baseline $B \neq 0$
- $f_{left} = f_{right} = f$
- 두 영상의 영상 평면은 동일한 평면 상에 있음

$$B : d = z : f$$

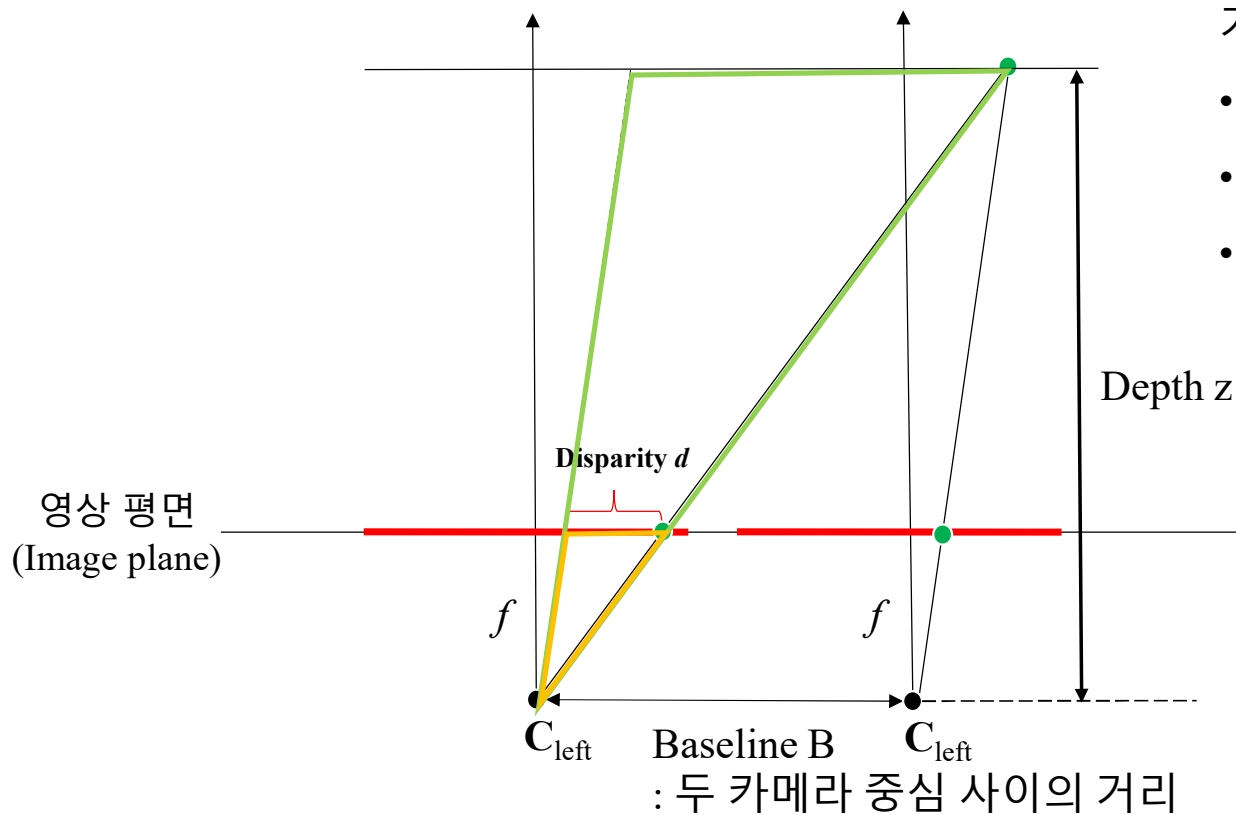
$$z = \frac{fB}{d}$$

$$z \propto \frac{1}{d}$$

$$z \propto fB$$

카메라 모델

- Disparity(양안시차)와 depth



가정

- Baseline $B \neq 0$
- $f_{left} = f_{right} = f$
- 두 영상의 영상 평면은 동일한 평면 상에 있음

$$B : d = z : f$$

$$z = \frac{fB}{d}$$

예) $f = 1000$, baseline $B = 65\text{mm}$, $d = 50$

$\rightarrow z = 1300 \text{ mm}$

카메라 모델

- 스테레오 영상 : 일반적으로 일치점들은 영상에서 동일한 높이에 있다고 가정

Left image



Right image

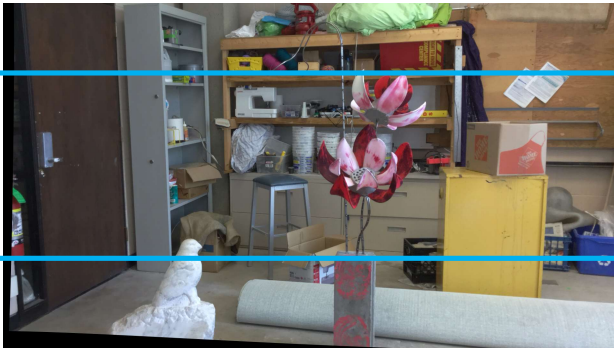


From <https://vision.middlebury.edu/stereo/data/>

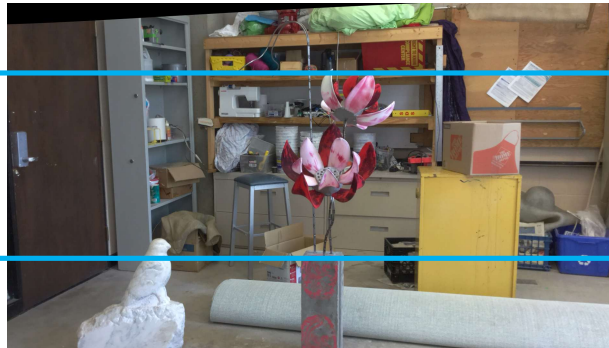
카메라 모델

- 스테레오 영상 : 일반적으로 일치점들은 영상에서 동일한 높이에 있다고 가정

Left image



Right image

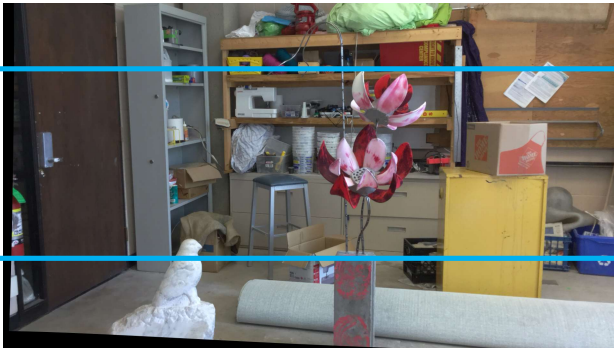


From <https://vision.middlebury.edu/stereo/data/>

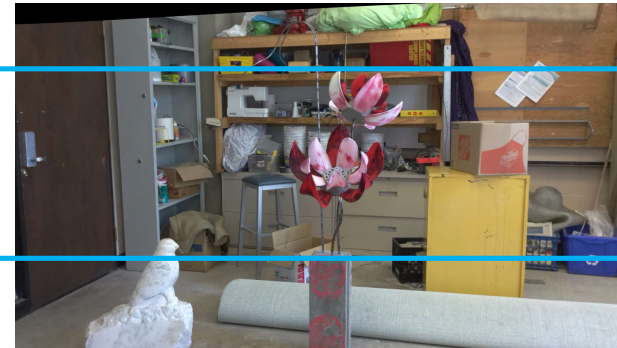
카메라 모델

- 스테레오 영상 : 일반적으로 일치점들은 영상에서 동일한 높이에 있다고 가정
- Disparity map(양안시차지도, 깊이지도, 깊이맵) : 스테레오 영상의 disparity를 픽셀 단위로 나타낸 map

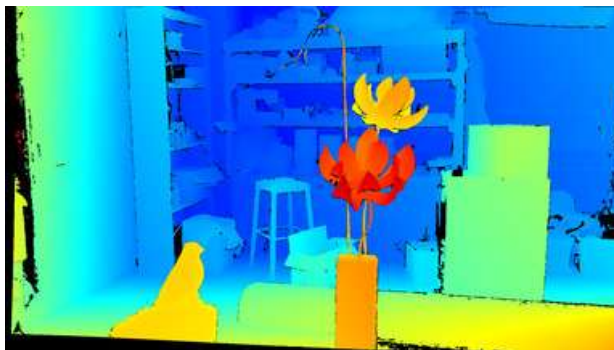
Left image



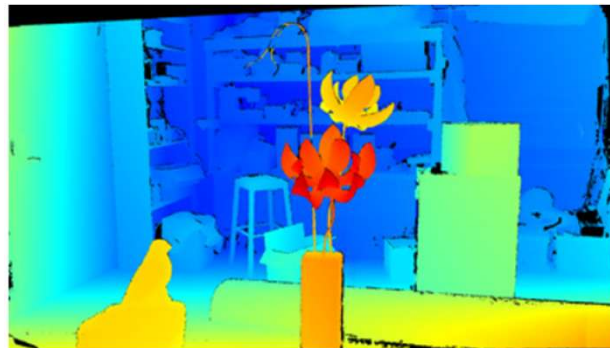
Right image



Disparity map of
left image



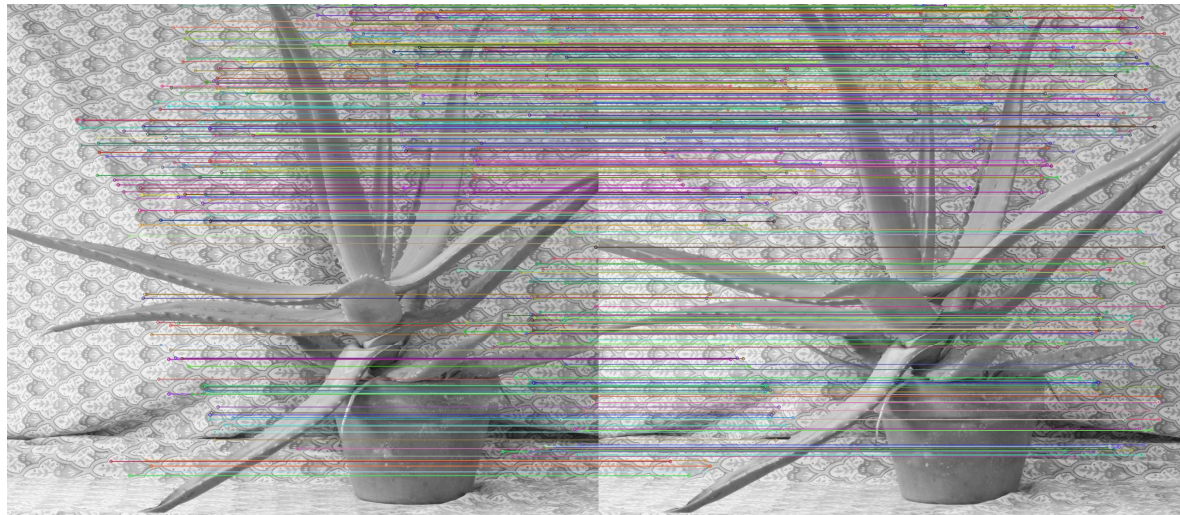
Disparity map of
right image



From <https://vision.middlebury.edu/stereo/data/>

Disparity Estimation(양안시차 추정)

- Disparity estimation(양안시차 추정) → depth estimation(깊이 추정)
- SIFT, ORB 등 특징 추출 기술을 이용하는 경우
→ 특징이 아닌 대부분의 픽셀의 disparity 추정이 어려움



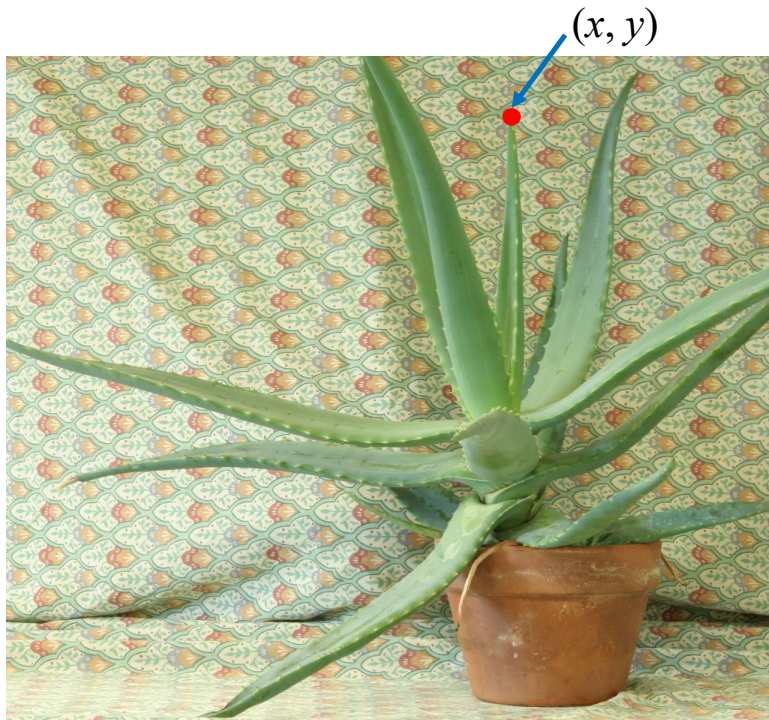
Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD, SSD, NCC 등
- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지를 최소화하는 방식으로 깊이지도 생성
 - GraphCut(그래프 기반 방법) 등
- Semi-global matching(세미-전역 매칭) 기반 방법
 - 블록 매칭의 속도와 전역 매칭의 정확도 사이에서 균형을 맞춘 방식
 - SGBM(Semi-Global Block Matching) 등
- Deep learning(딥러닝) 기반 방법
 - 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
 - RAFT-Stereo 등

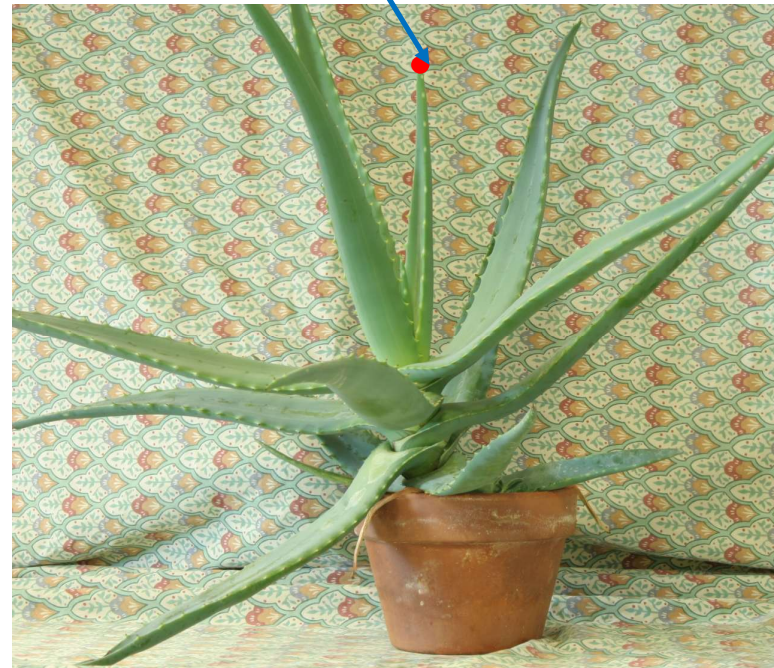
Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD, SSD, NCC 등

Left image



$(x-d, y)$



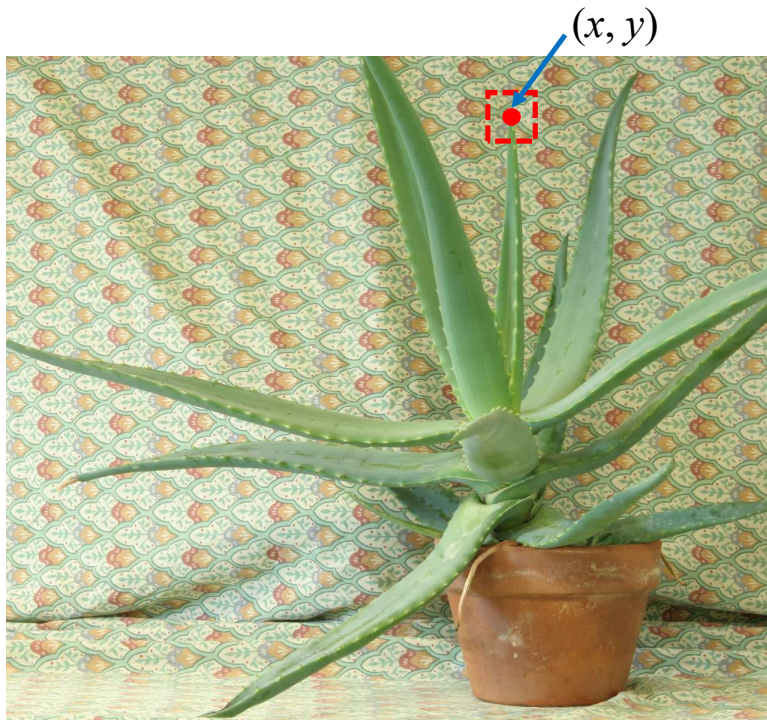
d : disparity

Right image

Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD, SSD, NCC 등

Left image



$(x-d, y)$



 : 블록

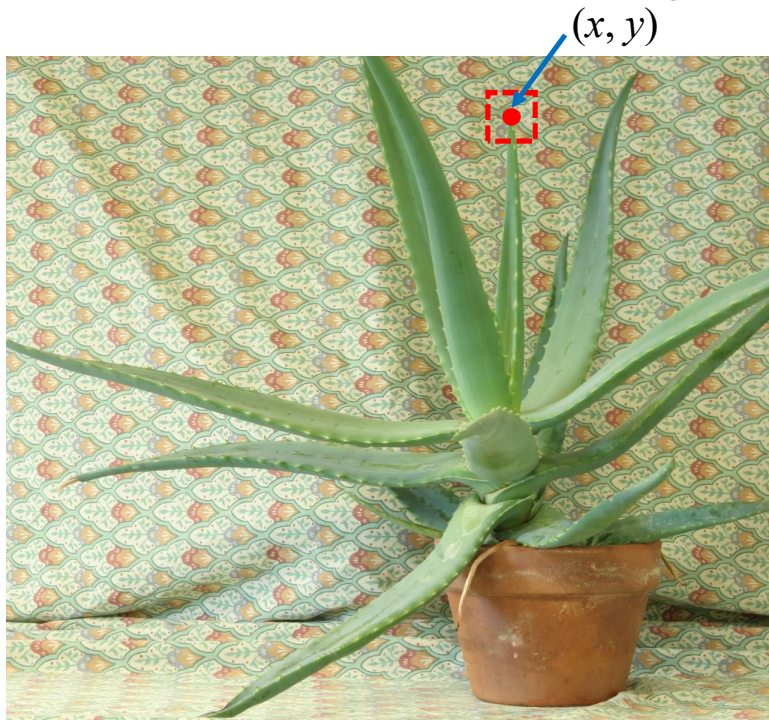
d : disparity

Right image

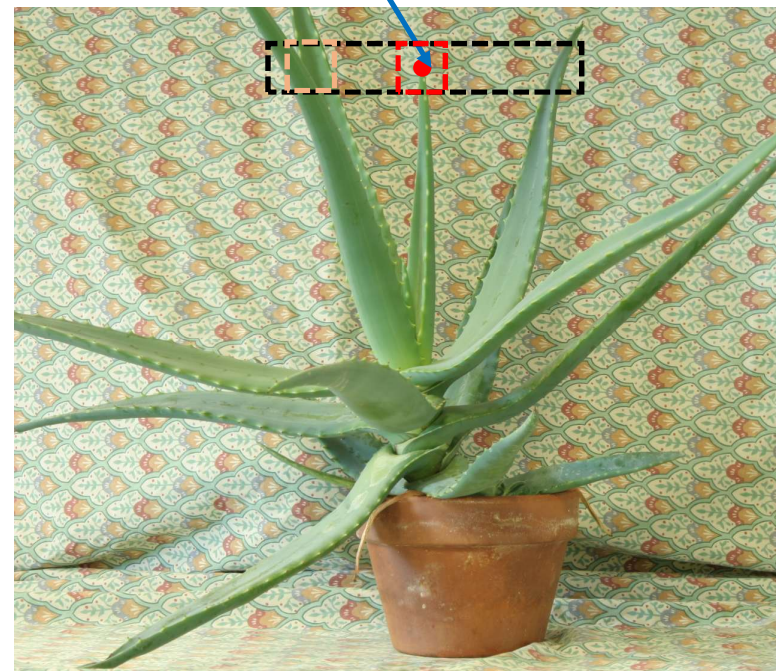
Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD, SSD, NCC 등
 - 주요 파라미터 : 블록사이즈, search range

Left image



$(x-d, y)$ [dashed box] : Search range



[red dashed box] : 블록

d : disparity

Right image

Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD(Sum of Absolute Difference)
 - 대응하는 픽셀 값 차이의 절댓값을 모두 더함
 - 연산량이 가장 적음

$$SAD(x, y, d) = \sum_{(i,j) \in W} |I_L(x + i, y + j) - I_R(x + i - d, y + j)|$$

- SSD(Sum of Squared Difference)
 - 픽셀 값 차이를 제곱하여 더함
 - 차이가 큰 픽셀(노이즈)에 대해 더 큰 페널티를 부여

$$SSD(x, y, d) = \sum_{(i,j) \in W} (I_L(x + i, y + j) - I_R(x + i - d, y + j))^2$$

3x3 인 블록(W)의 (i,j)예시

-1, -1	0, -1	1, -1
-1, 0	0, 0	1, 0
-1, 1	0, 1	1, 1

Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - NCC(Normalized Cross Correlation)
 - 분자에는 두 블록 사이의 공분산, 분모에는 각 블록의 표준편차의 곱이 들어감
 - $-1 \leq NCC \leq 1$ 이며, 1에 가까울수록 두 패치가 완벽하게 일치함을 뜻함
 - 두 블록의 상관계수(correlation coefficient)를 구하는 것과 같음

3x3 인 블록(W)의 (i,j) 예시

-1, -1	0, -1	1, -1
-1, 0	0, 0	1, 0
-1, 1	0, 1	1, 1

$$NCC = \frac{\text{Cov}(W_L, W_R)}{\sqrt{\text{Var}(W_L) \cdot \text{Var}(W_R)}}$$

두 블록 사이의 공분산

$$\text{Cov}(W_L, W_R) = \frac{1}{N} \sum_{(i,j) \in W} (I_L(x+i, y+j) - \bar{I}_L)(I_R(x+i-d, y+j) - \bar{I}_R)$$

각 블록의 표준편차의 곱

$$\sqrt{\text{Var}(W_L) \cdot \text{Var}(W_R)} = \sqrt{\frac{1}{N} \sum (I_L - \bar{I}_L)^2} \cdot \sqrt{\frac{1}{N} \sum (I_R - \bar{I}_R)^2}$$

Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - 블록 매칭의 장점
 - 알고리즘이 직관적이고 구현이 비교적 쉬움. 즉, 왼쪽 영상의 블록을 오른쪽 영상의 같은 행 (Scanline)을 따라 이동시키며 유사도(SAD, SSD 등)를 계산하기만 하면 됨
 - 연산 구조가 병렬 처리에 매우 유리함. GPU나 FPGA에서 수천 개의 블록을 동시에 계산할 수 있어 실시간(Real-time) 처리가 가능
 - 특정 탐색 범위 내의 데이터만 로드하면 되므로 메모리 점유율이 비교적 낮음

Disparity Estimation

- Block matching(블록 매칭) 기반 방법

- 블록 매칭의 단점

- 텍스처 부족 영역 (Textureless Region) 취약: 단색 벽면이나 하늘처럼 특징이 없는 곳에서는 모든 블록이 비슷함.
 - 반복 패턴 (Repetitive Patterns) 취약: 체커보드나 창문 격자처럼 같은 모양이 반복되는 곳에서는 어느 영역이 진짜 쌍인지 구분이 어려움
 - 가려짐 (Occlusion) 문제 취약: 한쪽 카메라에는 보이지만 다른 쪽 카메라에는 가려져서 보이지 않는 영역(Occlusion region)에 대해서는 블록 매칭으로 disparity 추정이 어려움



텍스처 부족 영역 (Textureless Region)

Disparity Estimation

- Block matching(블록 매칭) 기반 방법

- 블록 매칭의 단점

- 텍스처 부족 영역 (Textureless Region) 취약: 단색 벽면이나 하늘처럼 특징이 없는 곳에서는 모든 블록이 비슷함.
 - 반복 패턴 (Repetitive Patterns) 취약: 체커보드나 창문 격자처럼 같은 모양이 반복되는 곳에서는 어느 영역이 진짜 쌍인지 구분이 어려움
 - 가려짐 (Occlusion) 문제 취약: 한쪽 카메라에는 보이지만 다른 쪽 카메라에는 가려져서 보이지 않는 영역(Occlusion region)에 대해서는 블록 매칭으로 disparity 추정이 어려움



반복 패턴 (Repetitive Patterns)

Disparity Estimation

- Block matching(블록 매칭) 기반 방법

- 블록 매칭의 단점

- 텍스처 부족 영역 (Textureless Region) 취약: 단색 벽면이나 하늘처럼 특징이 없는 곳에서는 모든 블록이 비슷함.
 - 반복 패턴 (Repetitive Patterns) 취약: 체커보드나 창문 격자처럼 같은 모양이 반복되는 곳에서는 어느 영역이 진짜 쌍인지 구분이 어려움
 - 가려짐 (Occlusion) 문제 취약: 한쪽 카메라에는 보이지만 다른 쪽 카메라에는 가려져서 보이지 않는 영역(Occlusion region)에 대해서는 블록 매칭으로 disparity 추정이 어려움



가려짐 영역 (Occlusion region)

Disparity Estimation

- Block matching(블록 매칭) 기반 방법

- 블록 매칭의 단점

- 텍스처 부족 영역 (Textureless Region) 취약: 단색 벽면이나 하늘처럼 특징이 없는 곳에서는 모든 블록이 비슷함.
 - 반복 패턴 (Repetitive Patterns) 취약: 체커보드나 창문 격자처럼 같은 모양이 반복되는 곳에서는 어느 영역이 진짜 쌍인지 구분이 어려움
 - 가려짐 (Occlusion) 문제 취약: 한쪽 카메라에는 보이지만 다른 쪽 카메라에는 가려져서 보이지 않는 영역(Occlusion region)에 대해서는 블록 매칭으로 disparity 추정이 어려움



가려짐 영역 (Occlusion region)

Blocking Matching 실습

```
import cv2
import numpy as np
import time
import matplotlib.pyplot as plt
import urllib.request

# Aloe 이미지 다운로드
url1 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloreL.jpg'
url2 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloreR.jpg'
urllib.request.urlretrieve(url1, 'aloreL.jpg')
urllib.request.urlretrieve(url2, 'aloreR.jpg')

imgL_color = cv2.imread('aloreL.jpg')
imgL_gray = cv2.imread('aloreL.jpg', cv2.IMREAD_GRAYSCALE)
imgR_gray = cv2.imread('aloreR.jpg', cv2.IMREAD_GRAYSCALE)

# 원본 해상도 저장
orig_h, orig_w = imgL_gray.shape

# StereoBM 설정 (SAD 기반)
num_disp = 128
block_size = 15
```

```
stereo = cv2.StereoBM_create(numDisparities=num_disp, blockSize=block_size)

# disparity 계산
start_time = time.time()
disparity = stereo.compute(imgL_gray, imgR_gray)
elapsed_time = time.time() - start_time

# StereoBM 결과는 16배 스케일이므로 실제 픽셀 단위로 변환
disparity_float = disparity.astype(np.float32) / 16.0

# 시각화를 위한 정규화 (0~255)
# 시차 값이 0보다 작은 영역(무효값)은 제외
disparity_vis = cv2.normalize(disparity, None, 0, 255, cv2.NORM_MINMAX,
                              cv2.CV_8U)

# 결과 출력
print(f"이미지 크기: {orig_w}x{orig_h}")
print(f"소요 시간: {elapsed_time:.4f} 초")

plt.figure(figsize=(20, 10))
```

Blocking Matching 실습

```
# 왼쪽 원본 이미지
plt.subplot(1, 2, 1)
plt.title("Original Left Image")
plt.imshow(cv2.cvtColor(imgL_color, cv2.COLOR_BGR2RGB))
plt.axis('off')

# 원본 크기의 Disparity Map
plt.subplot(1, 2, 2)
plt.title(f"Full-Res SAD Disparity Map\n(NumDisp: {num_disp}, Block: {block_size})")
plt.imshow(disparity_vis, cmap='jet')
plt.colorbar(fraction=0.046, pad=0.04)
plt.axis('off')

plt.tight_layout()
plt.show()
```

Disparity Estimation

- 2.5D 영상
 - 2D 영상 + 각 픽셀의 깊이(Depth) 정보가 결합된 형태의 영상
- 2D, 3D 영상과의 비교
 - 2D 영상: 각 픽셀이 색상 정보(R, G, B)만 가짐
 - 3D 영상: 사물의 모든 면(앞, 뒤, 옆)에 대한 좌표(X, Y, Z) 및 색상 정보가 있어 자유로운 회전이 가능
 - 2.5D 영상 : 카메라가 바라보는 시점에서의 앞면 정보만 존재하되, 각 픽셀이 카메라로부터 얼마나 떨어져 있는지(깊이 값)를 알고 있는 상태



2.5D 영상의 개념

OpenGL 실습

- .ply
- 스탠퍼드 대학의 그래픽스 연구실에서 3차원 스캐너의 데이터를 효율적으로 저장하기 위해 개발한 포맷
- 각 정점(Vertex)에 더 많은 속성(Property)을 부여하는 데 특화되어 있음
- 예시)

```
ply
format ascii 1.0
element vertex 3
property float x
property float y
property float z
property uchar red
property uchar green
property uchar blue
element face 1
property list uchar int vertex_indices
end_header
0.0 1.0 0.0 255 0 0 # 1번 정점: 좌표(0,1,0), 색상(빨강)
-1.0 -1.0 0.0 0 255 0 # 2번 정점: 좌표(-1,-1,0), 색상(초록)
1.0 -1.0 0.0 0 0 255 # 3번 정점: 좌표(1,-1,0), 색상(파랑)
3 0 1 2 # 면: 3개의 정점을 사용하며, 인덱스는 0, 1, 2
```

2.5D 영상 실습

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 1. 이전 단계에서 생성한 데이터 준비 (imgL_color, disparity_float 사용)
scale = 1.0
img_small = cv2.resize(imgL_color, None, fx=scale, fy=scale)
disp_small = cv2.resize(disparity_float, None, fx=scale, fy=scale)

h, w = disp_small.shape

# 초점거리(f)와 베이스라인(B)은 임의의 값을 설정
f = 0.8 * w # 가상의 초점 거리
B = 1.0     # 가상의 카메라 간격

# 3. 3D 좌표 계산
x_coords, y_coords = np.meshgrid(np.arange(w), np.arange(h))

mask = disp_small > 0.5
```

```
# Z = (f * B) / disparity
z = np.zeros_like(disp_small)
z[mask] = (f * B) / disp_small[mask]

# X, Y 좌표 역투영
x = (x_coords - w/2) * z / f
y = (y_coords - h/2) * z / f

# 4. 시각화 데이터 준비 (유효한 점만 필터링)
points_x = x[mask].ravel()
points_y = y[mask].ravel()
points_z = z[mask].ravel()

# 색상 정보 추출 (BGR -> RGB 및 0~1 정규화)
colors = cv2.cvtColor(img_small, cv2.COLOR_BGR2RGB)[mask].reshape(-1, 3) / 255.0
```

2.5D 영상 실습

```
def save_point_cloud_ply(filename, points_x, points_y, points_z, colors):  
    """  
    포인트 클라우드 데이터를 PLY 파일로 저장  
    points_x, y, z: 1차원 넘파이 배열 (좌표)  
    colors: 1차원 넘파이 배열 (0~1 범위의 RGB 값)  
    """  
    # 색상을 0~255 범위의 정수로 변환  
    colors_int = (colors * 255).astype(np.uint8)  
  
    # 헤더 작성  
    num_points = len(points_x)  
    header = f"""ply  
format ascii 1.0  
element vertex {num_points}  
property float x  
property float y  
property float z  
property uchar red  
property uchar green  
property uchar blue  
end_header  
"""
```

```
# 데이터 결합 및 저장  
with open(filename, 'w') as f:  
    f.write(header)  
    for i in range(num_points):  
        # 좌표와 색상을 한 줄씩 기록  
        f.write(f'{points_x[i]} {points_y[i]} {points_z[i]} {colors_int[i, 0]}  
{colors_int[i, 1]} {colors_int[i, 2]}\n')  
  
    print(f'성공: '{filename}' 파일이 저장되었습니다. (총 {num_points}개의 점)')  
  
# 파일 저장 실행  
save_point_cloud_ply("aloe_2_5d.ply", points_x, points_y, points_z, colors)
```

2.5D 영상 실습

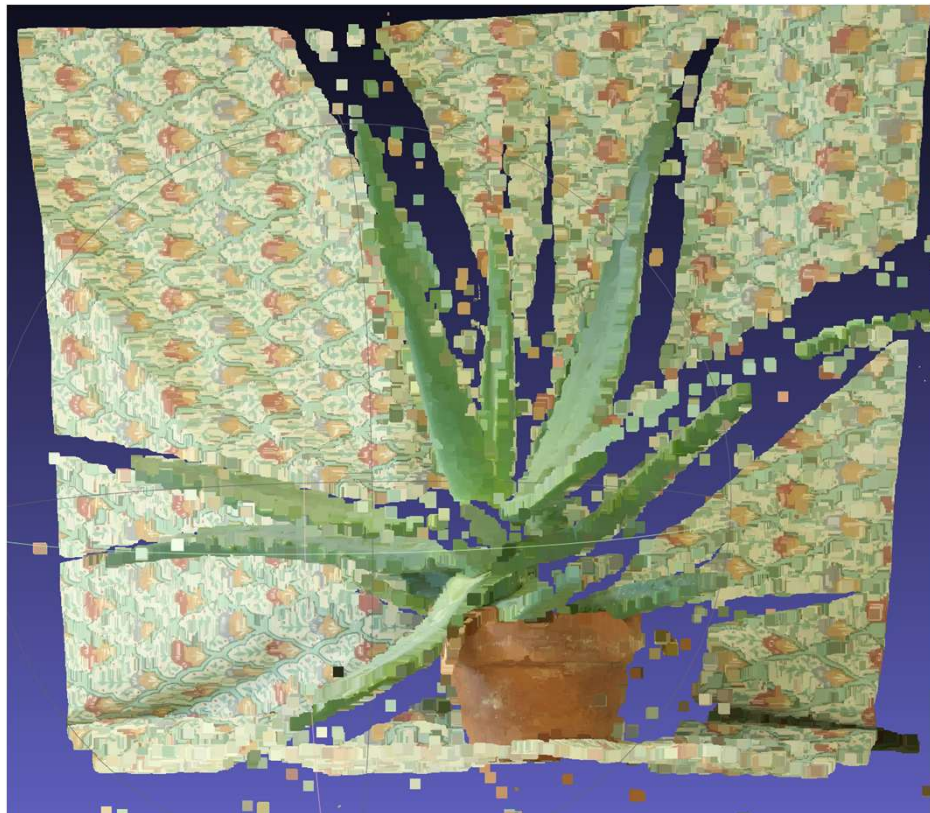
- Meshlab
 - 메쉬(Triangular Meshes)와 Point Cloud와 같은 3D 모델 데이터를 편집, 정리, 복원 및 렌더링 하기 위해 개발된 오픈 소스 소프트웨어
 - Ply, obj 포맷 데이터 확인 때 많이 사용
 - 다운로드 : <https://www.meshlab.net/>



장점	단점
무료 및 오픈 소스: 누구나 제한 없이 사용 가능.	메뉴가 비교적 복잡
강력한 필터: 수백 개의 수학적 필터 제공.	대용량 데이터 처리 시 가끔 튕김.
광범위한 포맷: PLY, STL, OFF, OBJ, 3DS 등 거의 모든 포맷 지원.	직접 모델링은 불가

2.5D 영상 실습

- 2.5D 영상(컬러영상 + disparity map) 예시



Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d)$$

- $E_{data}(d)$ (Data Term) : 왼쪽 영상의 픽셀과 오른쪽 영상의 대응 픽셀이 얼마나 유사한지를 측정(유사도)
- $E_{smooth}(d)$ (Smoothness Term) : 인접한 픽셀끼리는 d 값이 비슷해야 한다는 제약 조건을 부여함.

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d) \quad d : \text{깊이지도에서의 disparity(양안시차)들}$$

- $E_{data}(d)$ (Data Term) : 왼쪽 영상의 픽셀과 오른쪽 영상의 대응 픽셀이 얼마나 유사한지를 측정(유사도)
- 즉, 왼쪽 영상의 픽셀과 오른쪽 영상의 대응 픽셀 간 유사도가 가장 클 때 $E_{data}(d)$ 는 최소가 됨
- 일반적으로 블록 매칭 결과와 같음
- 예) $E_{data}(d) = \sum_{(x,y)} SAD(x, y, d)$
- $E_{data}(d)$ (Data Term) 만 있다면 블록 매칭과 동일한 문제 발생

Disparity Estimation

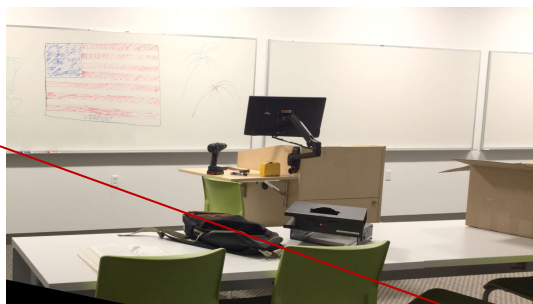
- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d)$$

- $E_{smooth}(d)$ (Smoothness Term) : 인접한 픽셀끼리는 d 값이 비슷해야 한다는 제약 조건을 부여함.
- 노이즈를 억제하고 매끄러운 깊이지도를 만들



텍스처 부족 영역
(Textureless Region)



매끄러운 깊이값을 가짐



반복 패턴
(Repetitive Patterns)



Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 깊이지도(disparity map, depth map)의 특징
 - 깊이가 변하는 지역을 제외하고 주변 픽셀의 d 값이 서로 비슷함(비슷한 깊이를 가지기 때문)

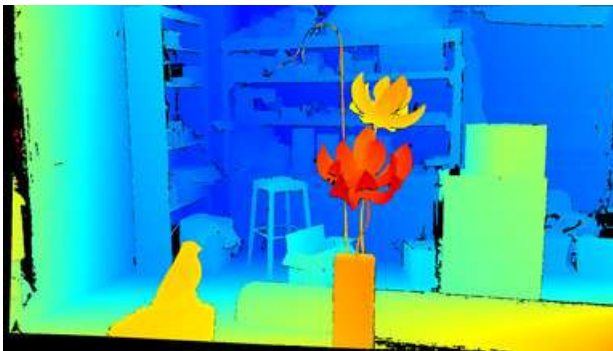
Left image



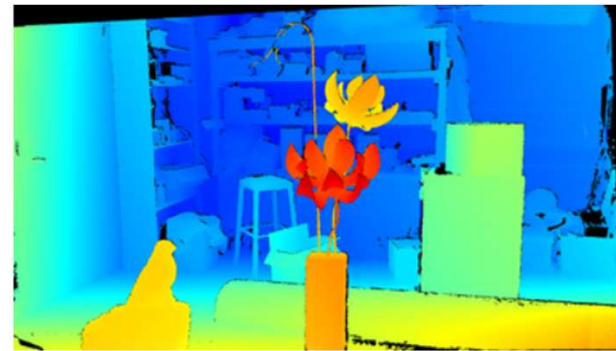
Right image



Disparity map of
left image



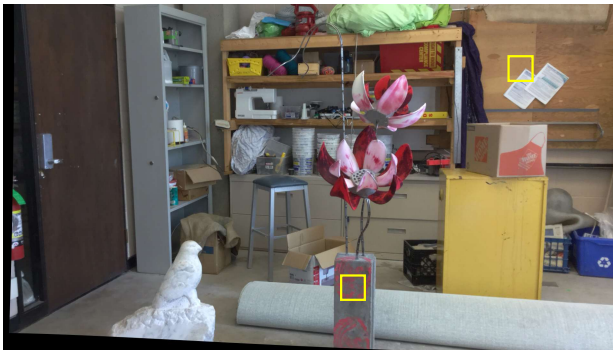
Disparity map of
right image



Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 깊이지도(disparity map, depth map)의 특징
 - 깊이가 변하는 지역을 제외하고 주변 픽셀의 d 값이 서로 비슷함(비슷한 깊이를 가지기 때문)

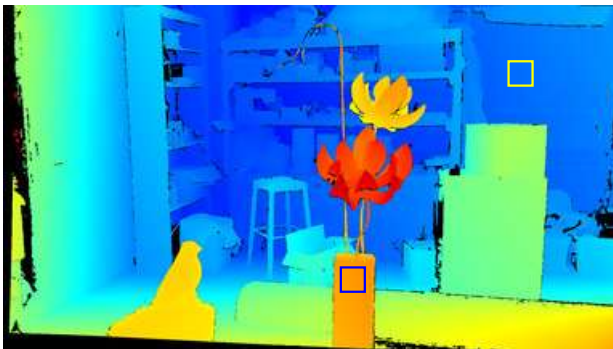
Left image



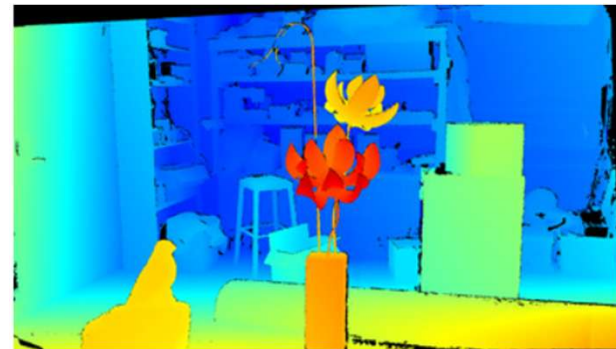
Right image



Disparity map of
left image



Disparity map of
right image



Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d)$$

- $E_{smooth}(d)$ (Smoothness Term) : 인접한 픽셀끼리는 d 값이 비슷해야 한다는 제약 조건을 부여함.
- 즉, 인접한 픽셀 간 d 값의 차가 작을수록 $E_{smooth}(d)$ (Smoothness Term)는 작아짐
- 만약 깊이지도의 모든 d 가 동일하면 $E_{smooth}(d) = 0$ (최소) 가 됨
- 예) 인접한 두 픽셀 p, q 의 disparity를 d_p, d_q 라고 하면

1) $E_{smooth}(d_p, d_q) = \lambda \cdot [d_p \neq d_q]$: p, q 의 disparity가 다르면 무조건 λ 만큼 에너지가 증가

2) $E_{smooth}(d_p, d_q) = \lambda \cdot |d_p - d_q|$: p, q 의 disparity의 차가 클수록 에너지가 더 많이 증가

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d)$$

- $E_{data}(d)$ 와 $E_{smooth}(d)$ 간에는 trade-off 관계가 있음
- $E(d)$ 를 최소화하는 것은 $E_{data}(d)$ 와 $E_{smooth}(d)$ 간의 최적의 균형을 찾는 것
- $E(d)$ 를 최소화하는 최적의 깊이지도 d_{opt} 를 찾는 것은 현실적으로 불가능하기 때문에 다양한 최적화 방법으로 d_{opt} 의 근사해를 구함

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - GraphCut
 - 깊이 지도를 픽셀 간 그래프로 모델링하고 2차원 최적화를 수행
 - Dynamic programming
 - 각 라인 별로 매칭 비용에 대한 테이블을 만들고 DP로 최적해를 구함(1차원 최적화)



Stereo Image
(Left image)



Ground truth (정답)
disparity map



Block matching
(SAD)



Global matching
(GraphCut)



Global matching
(Dynamic Programming)

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - Dynamic programming
 - 각 라인 별로 매칭 비용에 대한 테이블을 만들고 DP로 최적해를 구함(1차원 최적화)
 - 예) 영상의 크기가 10x8이고 search range가 5라고 하면

[illegible]

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - Dynamic programming
 - 각 라인 별로 매칭 비용에 대한 테이블을 만들고 DP로 최적해를 구함(1차원 최적화)
 - 예) 영상의 크기가 10x8이고 search range가 5라고 하면

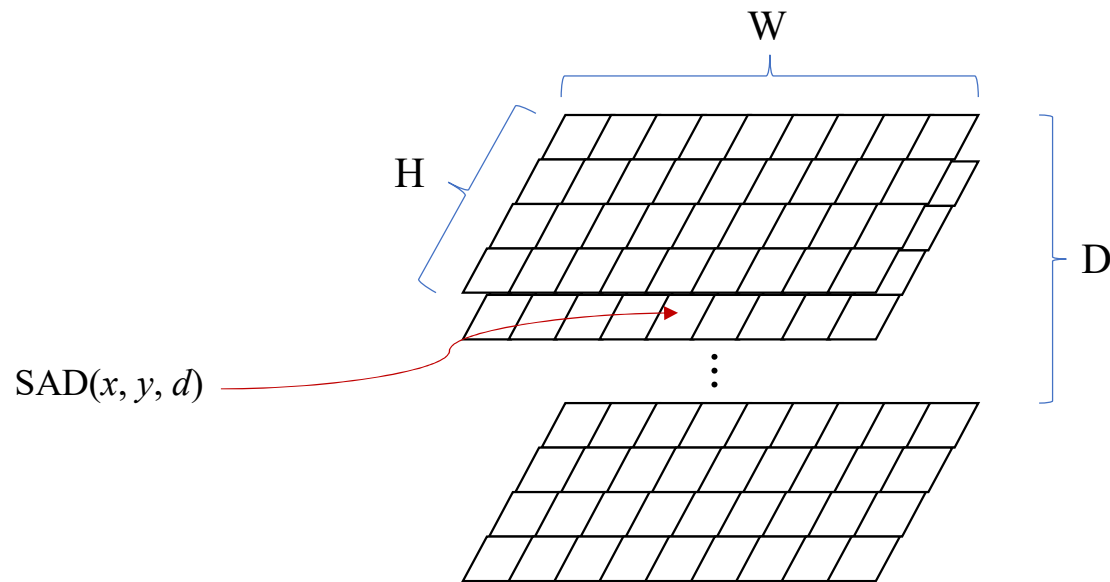
Search range {

SAD(0, 0)									
			SAD(x, d)						
									SAD(9, 4)

8개의 테이블에 대해 DP 수행

Disparity Estimation

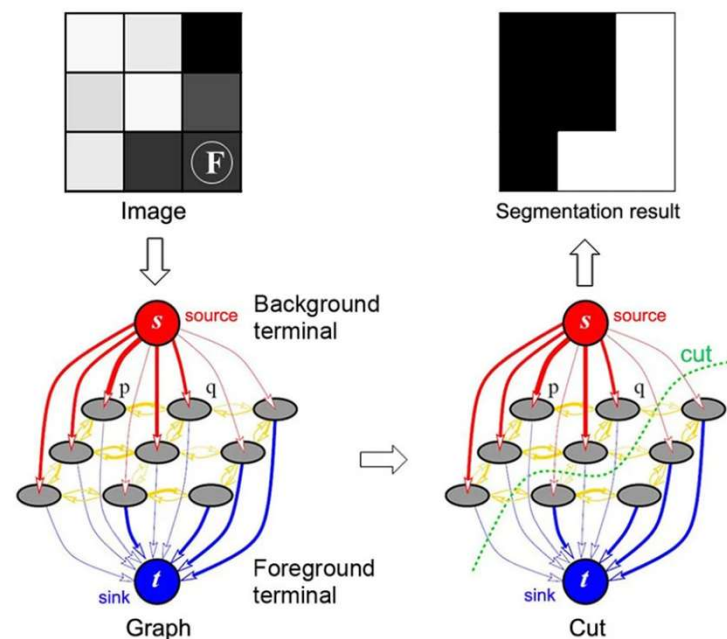
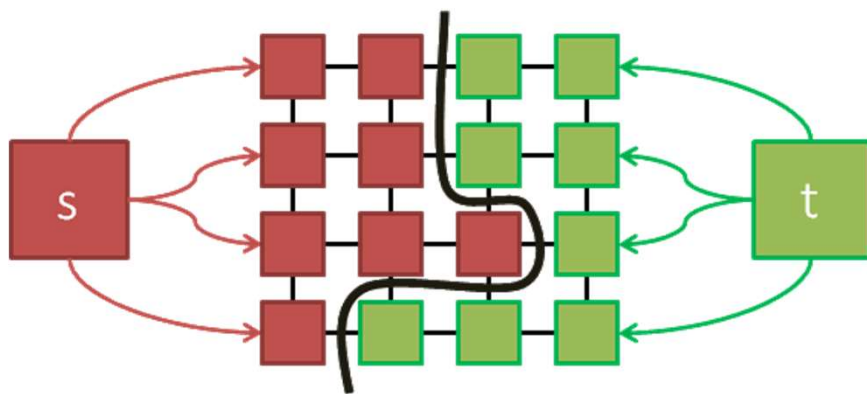
- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - GraphCut
 - 영상의 크기가 $W \times H$ 이고 search range가 D 라고 하면 $W \times H \times D$ 의 cost volume을 생성



cost volume의 예

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - GraphCut
 - 영상의 크기가 $W \times H$ 이고 search range가 D 라고 하면 $W \times H \times D$ 의 cost volume을 생성
 - Cost volume을 그래프로 모델링하고 minimum cut을 구함으로써 2차원 최적화를 수행

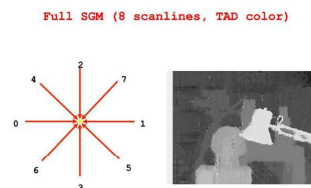
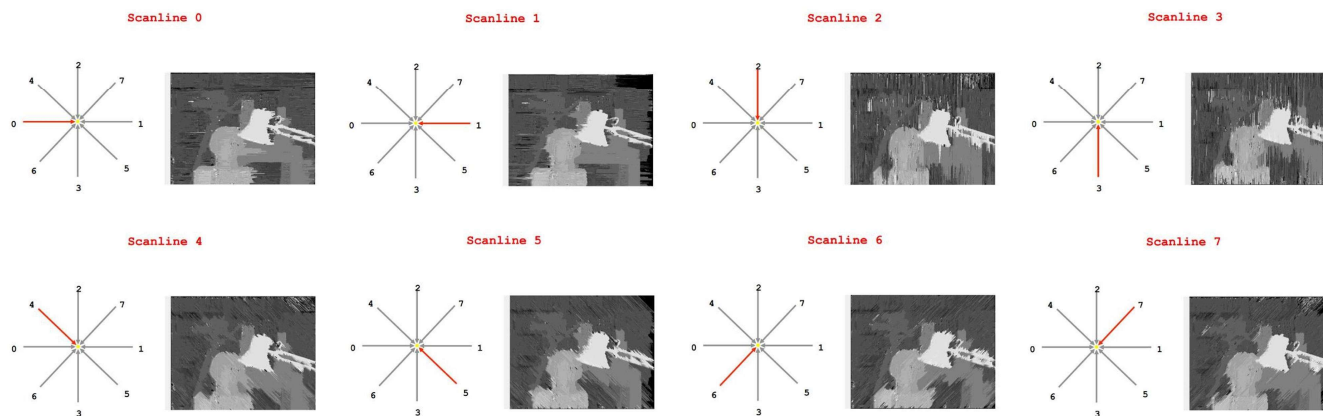
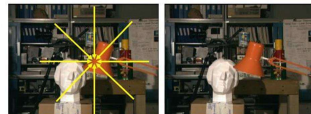


Disparity Estimation

- Semi-global matching(세미-전역 매칭) 기반 방법
 - 블록 매칭의 속도와 전역 매칭의 정확도 사이에서 균형을 맞춘 방식
 - Dynamic programming 기반 전역 매칭 방법은 한 라인에 대해서만 최적화를 수행하므로 라인 간 깊이 불일치로 인한 줄무늬가 생김
 - Semi-global matching에서는 8방향 또는 16방향에서 Dynamic programming을 수행하여 각각의 방향에 대해 경로 비용을 계산
 - 이후 모든 방향의 경로 비용을 더하고 이로부터 최적의 깊이 지도를 찾음

Disparity Estimation

- S



화를 수행하므로 라인 간 깊이 불

ing을 수행하여 각각의 방향에

음

Smoothness term의 예 : $C(p, d) = \begin{cases} 0 & \text{if } d_p = d_q \\ P_1 & \text{if } |d_p - d_q| = 1 \\ P_2 & \text{if } |d_p - d_q| > 1 \end{cases}$

Disparity Estimation

- Semi-global matching(세미-전역 매칭) 기반 방법
 - 블록 매칭의 속도와 전역 매칭의 정확도 사이에서 균형을 맞춘 방식
 - 장점
 1. 여러 방향의 정보를 취합하므로 기존 Dynamic Programming의 문제인 가로 줄무늬가 거의 나타나지 않음
 2. 각 경로 계산이 독립적이어서 GPU나 FPGA를 통한 병렬화에 매우 유리

Semi-Global Matching 실습

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
import urllib.request

# 데이터 로드 (ALOE 데이터셋 경로)
url1 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloreL.jpg'
url2 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloreR.jpg'
urllib.request.urlretrieve(url1, 'aloreL.jpg')
urllib.request.urlretrieve(url2, 'aloreR.jpg')
imgL = cv2.imread('aloreL.jpg')
imgR = cv2.imread('aloreR.jpg')

# StereoSGBM 파라미터 설정
window_size = 3
min_disp = 0
num_disp = 16 * 8 # ALOE 데이터의 최대 시차에 맞춰 16의 배수로 설정

# SGM 모드
# 3-Way (Fast) : cv2.STEREO_SGBM_MODE_SGBM_3WAY
# 5-Way (Default) : cv2.STEREO_SGBM_MODE_SGBM
# 8-Way (Full HH) : cv2.STEREO_SGBM_MODE_HH
```

```
stereo = cv2.StereoSGBM_create(
    minDisparity=min_disp,
    numDisparities=num_disp,
    blockSize=window_size,
    # P1, P2: Smoothness Term 가중치
    P1=8 * 3 * window_size**2,
    P2=32 * 3 * window_size**2,
    disp12MaxDiff=1,
    uniquenessRatio=10,
    speckleWindowSize=100,
    speckleRange=32,
    preFilterCap=63,
    mode=cv2.STEREO_SGBM_MODE_SGBM
)
# 깊이 지도 계산
disparity = stereo.compute(imgL, imgR).astype(np.float32) / 16.0

plt.figure(figsize=(12, 6))
plt.subplot(121), plt.imshow(cv2.cvtColor(imgL, cv2.COLOR_BGR2RGB)),
plt.title('Left Image')
plt.subplot(122), plt.imshow(disparity, 'jet'), plt.title('Disparity Map (SGBM)')
plt.colorbar()
plt.show()
```

과제 #2

1. 과제#1 이후부터 현재까지 실습 코드를 모두 수행하고 결과 출력 (2점)
 - 각 수행마다 코드에는 어떤 과정인지 주석 처리

제출 방법 : LMS로 압축 파일 제출

- 코랩에서 다음과 같은 이름을 가진 노트북 파일을 생성한다.

ComputerVision_2ndHW_Class본인소속반_학번_영문이름.ipynb

예) ComputerVision_2ndHW_Class1_24xxxxx_HanshinLim.ipynb

- 생성된 코랩 노트북 파일(.ipynb)에서 코드를 구현 및 결과를 출력한다. 결과 출력시 어떤 결과인지를 명시한다.
- Due :
 - 1반 : 5월 11일 밤 12시 (이후 제출 0점)
 - 2반 : 5월 13일 밤 12시 (이후 제출 0점)